DATA WAREHOUSING & BUSINESS INTELLIGENCE

Assignment 2

Integration, History & OLAP

Covers	Slowly changing dimensions, and what each type does to a report you already published · OLAP navigation on the finished star
Tasks	T1–T8 of the integration workbook, **and the hands-on lab**
Marks	**100** in the workbook + 15 in the paper quick check
CLO	CLO1, CLO2 and CLO3
Component	Hands-on Exercises (20% of the course)
Work	**Individual**
Tool needed	**Power BI Desktop, Power Query in Excel, or pandas**
Submit	A **GitHub repository link** on the LMS — folder `assignment2/`
Due	As posted on the LMS, before the midterm

What this assignment is

This is the last assessment before the midterm. The midterm is a pipeline and a star under time, and it will be built from the same four sources. Everything the course has taught is in this one assignment: the business question, mismatched files and the tiers, grain and surrogate keys, the star, named transform steps, the reject-route-remove identity — and one idea that is new, which is that a dimension changes and a published number must not.

The first ETL exercise handed over seventeen named steps. Assignment 1 handed over the targets and nothing else. **This one hands over the targets and a second month that already exists** — and the second month is the constraint.

Loading April correctly is the easy half, and you have done it twice already. The hard half is loading April without disturbing February or March, which is the first time in this course that *not* changing something has been worth marks.

The case

April closed, and three things happened at once.

A fourth carrier. PakFreight signed **ShaheenEx**, a courier partner that consolidates parcels and sub-contracts PakFreight's own trucks when a load fills. ShaheenEx does not file a spreadsheet. It files **JSON** — one object per consignment, weight in **grams**, status as an **integer code**, the customer nested inside the record, and a booking **timestamp** rather than a date. The three mismatched files are now four, and the fourth is not tabular at all.

The dimension moved. Six customers changed during April. Three moved head office across a provincial border, two were re-classified into a different sector, one was renamed after an acquisition. `customer_changes.csv` carries each change with an **effective date**.

The board asked for one table. Weight by the customer's head-office province, by month, February to April. March's Punjab figure was published in April at **7,566,850 kg**. After you load April, it must still read 7,566,850 kg.

That last sentence is the assignment. Overwrite the three relocated customers and every total still reconciles to the kilogram — and **883,350 kg of March's freight quietly changes province.** Nothing in the reject log catches it, nothing in the identity catches it, and the number the board already has in front of it is now wrong.

The data you need

Everything is in the folder distributed with this assignment.

File	What it is
`data/PakFreight_IntegrationLab_Starter.xlsx`	**Start here.** All eight tasks. Cream cells are yours to fill
`data/april/marine_april.csv`	**256 rows.** IDs as issued, dates `dd/MM/yyyy` as **text**, status UPPER CASE with stray spaces, kilograms
`data/april/rail_april.xlsx`	**318 rows.** Two title rows above the header, the `RL-` prefix stripped off every consignment number, weight in **METRIC TONNES**
`data/april/road_april.csv`	**739 rows.** Lower-case IDs, ISO dates, lower-case status with stray spaces, one blank line and one `TOTAL` footer
`data/april/courier_april.json`	**492 records. The new one.** Grams, integer status codes, nested `pickup` / `dropoff` / `charge` / `customer`, `booked_at` timestamps with a `+05:00` offset
`data/customers_master.csv`	The registry, 46 rows and 40 businesses — unchanged
`data/customer_changes.csv`	**The six changes, with effective dates.** The whole of T5 starts here
`data/alias_lookup.csv`	The alias map, **unchanged**: 23 raw strings to 14 cities. April contains strings it has never seen
`data/dims/*.csv`	The star carried forward. `DimDate` covers **Q1 only**, `DimRoute` has **20 lanes** and `DimMode` has **three modes**. April needs more of all three
`data/carryforward/FactShipment_February.csv`	**34 rows, 578,650 kg.** February's one number, as a fact table
`data/carryforward/FactShipment_March.csv`	**1,200 rows, 17,542,050 kg.** The March load, verified, so nobody is blocked by the previous assignment
`data/carryforward/DimCustomer.csv`	**41 rows.** The customer dimension as it stands. It has **no history columns at all** — that is the point

There is no `FactShipment_April`, no `RejectLog_April` and no integrated fact table. Producing all three, without moving February or March, is the assignment.

There is no lab guide, by design, for the second assessment running. You know how to shape a file, rename to a contract, trim, cast, conform, key and reject. A JSON source is the same five steps with a different first one.

Power BI Desktop, Power Query inside Excel (**Data > Get Data > From JSON**), or **pandas** — all three earn full marks, say which you used. Every formula in the workbook is `COUNTIF`, `COUNT`, `SUM`, `SUMPRODUCT` and `IF`.

The contract

One target table, nine columns. Every source is mapped onto it before anything else happens.

Column	Rule
`ShipmentID`	non-blank, trimmed, upper case; degenerate dimension
`ShipDate`	parses to a date **inside April 2026**; truncated to the day
`OriginCity` / `DestCity`	resolves through `alias_lookup.csv`, else `-1`
`WeightKG`	numeric, greater than zero, not heavier than the mode's heaviest vehicle
`FreightCostPKR`	numeric; **may be negative** — that is a credit note
`Status`	maps to a `DimStatus` member, else `-1`
`CustomerID`	resolves to a business in the registry, else `-1`
`SourceMode`	Marine, Rail, Road or Courier

Mode capacities, for the weight rule: **Marine 25,000 kg · Rail 35,000 kg · Road 8,000 kg · Courier 8,000 kg** — the courier is 8,000 and not 1,000 because ShaheenEx sub-contracts PakFreight's trucks, which is also why fifteen rows of this month's data are the same consignment filed twice.

The rules

- **Extract corrects nothing.** Grams stay grams and tonnes stay tonnes until the transform layer. 1,805 rows arrive and 1,805 rows land.
- **Nothing is dropped silently.** A row that cannot be loaded goes to `RejectLog_April` **with a reason, a source and a locator**. A row whose lookup fails goes to the Unknown member at `-1`, loads, and is **counted**.
- **A published number never moves.** If loading April changes a February or March figure, you have made a mistake, not an update.
- **Reject, route to Unknown, and remove as duplicate are three different decisions.** So is *leave history alone*.
- **The totals must not move.** Cleaning data never changes what the business shipped.
- **A total is not a row count.** Four of the wrong answers here reconcile on one and not the other, and one reconciles on both.
- **Every number in the workbook came out of the files in `data/`.** So can yours.

T1 · The warehouse, end to end · 10 marks

Part A — place each item · 6 marks

Ten items. For each: which tier it belongs to — **Source**, **Staging**, **Storage**, **Presentation** or **BI Application** — and one line on why.

The items are in the sheet. Four of them are things this assignment creates and did not exist before; one of them belongs to a tier most students will not pick first.

Part B — two written answers · 4 marks

1. A fourth source arrives. Name one thing that **must** change at each of the four tiers, and one thing that **must not**.
2. Why does the grams-to-kilograms conversion belong in the transform layer rather than in extract? Answer in terms of what you lose when the load has to be re-run.

> The tiers have been argued from a slide up to now. Here a real fourth source arrives, and the argument is worth marks only if it tells you where to put the conversion.

T2 · Profile four sources against one contract · 12 marks

Part A — the mapping · 4 marks

Nine contract columns. For each, say what all four sources call it and what has to happen to it. Three of the nine need something done to them in **every** source. One of them exists in only one source and has to be invented for the other three.

Part B — profile the staging table · 5 marks

Staging is **1,805 rows**: shaped and renamed, nothing transformed. Profile all eight data columns **before** you change a single value — blanks, distinct raw values, rows that **fail the stated rule**, and rows that merely **need a text transform**.

Self-check (in the sheet): the eight rule-failure counts sum to **74**, and exactly **74 distinct rows** fail at least one rule.

> Those two numbers being equal is a fact about this month, not a law. In March they differed. Check it rather than assume it — and then reconcile the 74:
>
> ```
> 74  =  35 rejected  +  38 routed to Unknown  +  1 removed as a duplicate
> ```
>
> The last term is the interesting one. A row can fail a rule and still never reach the fact table, for a reason that has nothing to do with the rule it failed.

Part C — three written answers · 3 marks

1. `courier_april.json` carries **91 distinct times of day**. The grain of `FactShipment` is one consignment per day. What do you do, and what goes wrong if the timestamp is keyed straight into `DateKey`?
2. Which column's distinct-value count most overstates how dirty it is, and why?
3. `road_april.csv` ends with a `TOTAL` footer whose weight and cost are **arithmetically correct** — 4,660,200 kg and PKR 34,970,000, which is exactly the sum of the rows above it. Why is it still not a row?

> The footer being right is worse than the footer being wrong. A wrong footer is caught by the identity; a right one is caught only by knowing what a row is.

T3 · Reject, route, remove · 14 marks

Part A — classify every class of row · 6 marks

Fifteen classes and four verdicts: **Load**, **Route to Unknown**, **Reject**, **Remove as duplicate**.

- **Reject** — the row cannot be loaded at all, and goes to `RejectLog_April` with a reason.
- **Route to Unknown** — the row loads, a lookup failed, the key is `-1`, and the count is published.
- **Remove as duplicate** — the row is not a second shipment, so taking it out restores the grain.
- **Load** — there is nothing wrong with it.

Self-check (in the sheet): 35 rejected · **38** routed to Unknown · **70** removed as duplicates · **2** removed at extract as structural · everything else loads.

The reject classes, and how many rows are in each:

Class	Rows
Weight is text — `approx 7000`, `TBC`, `--`, `7.2 kg`, `null`	10
Date does not parse, or is not in April 2026	7
Weight is negative	6
`ShipmentID` is blank	5
Cost is blank or text — `see invoice`, `null`	4
Weight is heavier than the mode's heaviest vehicle	3
Total	**35**

> The cost column is new. It never failed in February or March, and a pipeline that only ever validated weight loads four rows this month with no price on them. Nothing downstream will tell you: `SUM` skips a null and the weight total stays perfect.
>
> **Three rows look exactly like faults and are not.** A negative *weight* is impossible and is rejected; a negative *cost* is a credit note, and you have already met one in the Superstore log. Reject on "any negative" and you delete three real shipments.

Part B — specify the `RejectLog_April` schema yourself · 5 marks

List the columns you would give it and say what each one is for. The test is whether somebody who has never seen your pipeline can take a row out of that table, fix the source, and put it through again.

Six or seven columns is a good answer. Four is thin. Ten means you are storing the same thing twice.

Part C — three written answers · 3 marks

1. A CSV reject carries a line number. **What is the locator for a rejected record inside `courier_april.json`**, and why is the array index not quite enough on its own?
2. Rejecting a row and routing it to Unknown both mean "we could not resolve this". Why are they not the same decision, and what does each cost you if you choose the other?

3. **Fifteen rows are the same consignment filed by two carriers. Nine further pairs of rows share a business key and are genuinely different shipments.** State the de-duplication rule that removes exactly the fifteen and keeps all eighteen of the others.

Question 3 is the previous assignment's lesson arriving with the opposite sign. Last month a business key destroyed real history and the answer was *don't use one*. This month a business key is the only thing that finds a duplicate filed under two different IDs. The answer is not "use it" or "don't" — it is the scope you put on it.

T4 · The reconciliation identity, four sources · 12 marks

Part A — the ladder · 6 marks

Fill it in **from the files**.

Stage	Rows	SUM(WeightKG)	SUM(FreightCostPKR)
`marine_april.csv`, as extracted	?	?	?
`rail_april.xlsx`, as extracted	?	?	?
`road_april.csv`, as extracted	?	?	?
`courier_april.json`, as extracted	?	?	?
staging, all four appended	?	—	—
rejected	?	—	—
removed as duplicate — same ID, same file	?	—	—
removed as duplicate — same consignment, two carriers	?	—	—
loaded into `FactShipment_April`	?	**17,915,700**	**158,037,000**

The four per-file weight figures do not add up to anything until two units are fixed, and `SUM` silently skips the ten text weights in three of the four files. Work out why before you write them in.

Then the identity, which must come out at exactly zero:

```
staging  −  loaded  −  rejected  −  removed  =  0
1,805    −  1,700   −    35      −    70     =  0
```

Any other number means rows left your pipeline and nothing recorded where they went. That is still the only unforgivable outcome. A wrong number you can find; a missing row you cannot.

Part B — read the gap · 6 marks

Four students hand in four different answers. **Every one of them ran a pipeline that completed without an error.** For each: what did they do, and which check on the T8 sheet catches it?

```
1,710 rows   17,915,700 kg      PKR 158,037,000
1,691 rows   17,882,150 kg      PKR 157,724,000
1,682 rows   17,734,700 kg
1,700 rows    8,979,894.75 kg
```

The first is the sharpest thing on this sheet, and it is the same trap that was on it last month: the weight is right to the kilogram, the cost is right to the rupee, and ten rows in that fact table are not shipments.

The second destroys **nine real consignments** and sounds more rigorous than the right answer, which is exactly why it survives review. Name two of the nine.

The fourth has the row count perfect and is missing roughly half the freight in the country. One division, one place, and the cost per kilogram reads 17.60 instead of 8.82 — which is the cheapest check in this entire course.

T5 · Slowly changing dimensions · 18 marks

This is the heart of the assignment. Read `customer_changes.csv` first.

Change	Customer	What changed	Effective
1	CU-105 Ravi Pharma Ltd	HQ city **Lahore to Karachi** (Punjab to Sindh)	2026-04-07
2	CU-120 Gujranwala Fans Ltd	HQ city **Gujranwala to Islamabad** (Punjab to Islamabad ICT)	2026-04-12
3	CU-116 Swat Timber Traders	HQ city **Peshawar to Lahore** (KP to Punjab)	2026-04-19
4	CU-123 Clifton Electronics	Sector Electronics to Retail	2026-04-05
5	CU-132 Port Qasim Bulk Handlers	Sector Logistics to Marine Services	2026-04-21
6	CU-140 Keamari Cold Storage	Renamed **Keamari Cold Chain (Pvt) Ltd**	2026-04-15

Part A — the four types · 4 marks

For Type 0, Type 1, Type 2 and Type 3: what it does to the dimension row, how many rows `DimCustomer` holds after all six changes, whether you can still answer *"which province was this customer in when the shipment moved?"*, what March's Punjab figure reads afterwards, and when it is the right choice.

Part B — the board table, twice · 6 marks

`SUM(WeightKG)` for **March**, by the customer's head-office province. Once as it was published, and once after a Type 1 overwrite.

Province	As published	After Type 1
Balochistan	?	?
Islamabad ICT	?	?
Khyber Pakhtunkhwa	?	?
Punjab	**7,566,850**	?
Sindh	?	?
Total	**17,542,050**	**17,542,050**

Self-check (in the sheet), two figures and they are not the same:

March freight that changed province — the three relocated customers' March weight	**883,350 kg**
Net movement the Difference column shows — the sum of the positive differences	**552,900 kg**

One customer moved **into** Punjab while two moved out, so the province column nets off a third of the damage. Say which customer, and say why the net figure is the wrong one to report.

Both columns total 17,542,050 kg. The total is right to the kilogram either way, the identity holds either way, the reject log is empty of it either way. That is the whole problem, and it is why this is an eighteen-mark task and not a definition.

Islamabad ICT is the one to look at. Its March figure rises by 79 per cent — from 340,650 kg to 609,150 kg — because one customer's registered office moved. The province it moved out of loses 2.9 per cent, which is small enough that nobody queries it. That asymmetry is why this failure survives review: the error is glaring in the small province and invisible in the large one.

Part C — one decision per change · 4 marks

For each of the six changes, and for the `SignedUp` date: the type you would use, why, and whether it moves a published number.

They are not all the same answer. One of them is a Type 1 and arguing for Type 2 on it is wrong, not cautious. One of them has a reasonable case for Type 3.

Part D — build it · 4 marks

Convert `data/carryforward/DimCustomer.csv` to **Type 2**: `ValidFrom`, `ValidTo`, `RowIsCurrent`, a new surrogate `CustomerKey` for each new version, and the same `CustomerID` on both.

	Expected
Rows, all versions	**47**
Rows where `RowIsCurrent = Yes`	**41**
Rows where `RowIsCurrent = No`	**6**
Distinct `CustomerID` values	**40**

The three city moves split April in two. Read the counts off your own model and write them in: April rows before the effective date, April rows on or after it, and the kilograms on each side, for CU-105, CU-120 and CU-116.

> This is the part that is genuinely new. The customer lookup is no longer `CustomerID` to `CustomerKey`. It is `CustomerID` **and the shipment date** to `CustomerKey`, and getting it wrong does not raise an error — it puts a handful of early-April shipments in the wrong province and leaves every total correct.

Written, 1 of the 4 marks: a Type 2 row has a new surrogate key and the same `CustomerID`. Why must `FactShipment_March` keep pointing at the **old** key, and what breaks the day somebody "tidies up" by re-pointing it at the new one?

T6 · OLAP navigation · 14 marks

One model, February to April, **2,934 rows**. Five operations.

Part A — the cube, three ways · 5 marks

Ten figures, read off your own integrated model. The sheet checks every one.

Operation	What it means here
Roll-up	city to province to region, on the destination
Drill-down	April, destination Punjab, down to the city
Slice	one mode, all months
Dice	two modes x two provinces x one month
Pivot	mode against month

Part B — the pivot, and the number that lies · 4 marks

Cost per kilogram, **mode against month**, all twelve cells plus the margins.

Then the written part, which carries 3 of the 4 marks. Blended cost per kilogram was **8.31** in February and **8.38** in March. April reads **8.82**. Strip one mode out and April reads **8.46**.

Which mode, and what is the correct thing to tell a board that has been shown a rising cost-per-kg trend line?

> This is the course's opening question — *which categories lost money this week?* — with the whole machinery behind it. The trend line is not wrong. It is a true statement about a blend whose composition changed, presented as a statement about prices, and the fix is not a better chart. It is a sentence.

Part C — the aggregate table · 2 marks

Rows in a province x month aggregate over February to April, rows in the fact table it summarises, how often it must be rebuilt, and **what a Type 2 customer change does to it, and how far back**.

> The last one is where T5 and T6 meet. An aggregate is a cached answer. A Type 2 change does not invalidate last month's fact rows — but it does invalidate the cached answer, for every period the changed customer ever shipped in.

T7 · Selectivity, the index and the aggregate · 12 marks

Part A — selectivity · 5 marks

Selectivity = distinct values / rows. Compute it for all eight candidate columns from your own 2,934-row fact table: `DateKey`, `OriginCityKey`, `DestCityKey`, `RouteKey`, `ModeKey`, `StatusKey`, `CustomerKey`, `ShipmentID`. Then say, for each, whether the dashboard filters on it never, sometimes, or on every page.

Part B — the decision · 4 marks

1. Which single column do you index, and **why that one** rather than the most selective column or the one filtered on every page?
2. State the cost on write **as a number**: rows per load x indexes maintained, for your April load.
3. The February-to-April page is slow and the fact table is 2,934 rows. Name two causes an index cannot fix, and say which one **this dataset actually has**.

> `ShipmentID` is perfectly selective — 1.0000 — and nobody filters a dashboard on it. `ModeKey` is on every page and has four distinct values. Neither is the answer, and saying why is the task.

Part C — the join path · 3 marks

One business question — **total weight by province, February to April** — down three paths. Probes are fact rows x joins traversed.

Path	Tables	Joins	Probes
Star: `FactShipment` to `DimCity`	?	?	?
Snowflake: `Fact` to `DimCity` to `DimProvince` to `DimCountry`	?	?	?
Province x month aggregate, read directly	?	?	?

Star against snowflake was an argument, then it was arithmetic. Here there is a third row, it beats both by three orders of magnitude, and one mark is for the reason you would **still not build it yet**.

T8 · Definition of done · 8 marks

Refresh your model, read each number off it, type it in. **All twelve must read "pass".** Submitting with a single `fix` loses all 8 marks.

#	Check	Expected
1	Rows extracted into staging, all four files	**1,805**
2	Rows in `RejectLog_April`	**35**
3	Rows removed as duplicates	**70**
4	Rows in `FactShipment_April`	**1,700**
5	`SUM(WeightKG)`, April	**17,915,700 kg**

#	Check	Expected
6	`SUM(FreightCostPKR)`, April	**PKR 158,037,000**
7	April rows on the Unknown **city** member (`-1`)	**18**
8	April rows on the Unknown **status** member (`-1`)	**9**
9	April rows on the Unknown **customer** member (`-1`)	**11**
10	Rows in `DimCustomer`, all versions	**47**
11	Rows in `DimDate`	**120**
12	**Punjab, March, read off the integrated model**	**7,566,850 kg**

Checks 1 to 4 are the identity. They must satisfy `1 = 2 + 3 + 4` before any of the others mean anything.

Check 11 cannot be passed by cleaning April. `data/dims/DimDate.csv` is Q1 — ninety gapless rows ending on 31 March. April has thirty days and nowhere to put them. Extend it.

Check 12 is the only check in this course that is passed by not doing something. It is not about April at all.

Checks 7, 8 and 9 are the character question · 4 of the 8 marks

They expect **18**, **9** and **11**. Not zero. They are the only three checks on the sheet satisfied by **admitting something**.

So the sheet asks *how you know* — name the six unresolved city spellings, the five unresolved status values and the five unresolved customer references, and say what you would propose to the business for each group next month.

> Zero was available. Delete those rows and April reads 1,662, every other check still looks reasonable, and **383,600 kg** of freight PakFreight actually moved is gone from the quarter. Nobody would have found it.
>
> Two of the five unresolved statuses are integers, which is the only genuinely new shape of this problem here: `7` and `9` are not dirty data. They are perfectly clean values from a code table the warehouse has never been told about, and the right answer involves a conversation, not a transform.

The lab · The integrated load

No walkthrough. The deliverables are five tables and the arithmetic that ties them together:

- **`FactShipment_April`** — 1,700 rows, seven keys, one degenerate dimension, two additive facts, the Unknown member at `-1` used where it is needed.
- **`RejectLog_April`** — 35 rows, the schema you specified in T3, every row naming its source, its locator and its reason.
- **`DimCustomer`** — Type 2, 47 rows, 41 of them current.
- **`DimDate`** — 120 rows, gapless, Q1 and April.
- **`FactShipment_FebToApril`** — 2,934 rows, **36,036,400 kg**, **PKR 309,882,000**.

And two dimensions that grow rather than change:

- **`DimMode`** — Courier added. Four modes and Unknown.
- **`DimRoute`** — **24 lanes and Unknown.** April ships on four lanes the original star never saw, and **197 April rows** have no route key until you add them. After you do, 18 April rows remain on `-1`, and those are the unresolved cities, not missing lanes.

> A new mode and a new lane are **inserts**, not changes. Nothing historical is affected and no version row is needed. Telling an insert apart from a Type 2 change is half of the slowly-changing-dimension lecture.

If you finish early

Run the whole thing again without deleting anything. A second refresh must produce **1,700** April rows and not 3,400, and `DimCustomer` must still have **47** rows and not 53. A Type 2 loader that appends a new version

every time it runs is the most common way this goes wrong in production, and it looks completely correct the first time.

Marking guide

Task	What is marked	Marks
T1	Ten items placed across the five tiers, with a reason each (6) + what must and must not change at each tier, and where the unit conversion belongs (4)	**10**
T2	The nine-column mapping across four sources (4) + the staging profile, 74 failures and 74 distinct rows (5) + the timestamp, the overstating column and the correct footer (3)	**12**
T3	Fifteen classes against four verdicts (6) + the `RejectLog_April` schema you specify (5) + the JSON locator, reject-versus-route, and the scoped dedup rule (3)	**14**
T4	The ladder across four sources and the identity at zero (6) + four wrong answers read back to their cause and their check (6)	**12**
T5	The four types compared (4) + the board table twice, with 883,350 and 552,900 distinguished (6) + a type decision per change (4) + the Type 2 build, 47 / 41 / 6 / 40, and why March keeps the old key (4)	**18**
T6	Roll-up, drill-down, slice, dice and pivot (5) + cost per kg, and the sentence you say to the board (4) + the aggregate and what Type 2 does to it (2)	**14**
T7	Selectivity for eight columns with filter frequency (5) + the index decision, costed as a number, and two causes an index cannot fix (4) + three join paths (3)	**12**
T8	All twelve checks reading **pass** (4) + checks 7, 8 and 9 answered with *how you know*, and the sixteen raw values named (4)	**8**
	Workbook total	**100**
	Paper quick check, in class	15
Lab	The integrated load: five tables, both months undisturbed, every step named	*gate*

T5 is the heaviest task in the course so far, and check 12 is the heaviest single check. Both exist for the same reason: a failure that every other mechanism in this course would pass.

The lab is a gate, not a score. A workbook cannot have passed T8 honestly without the pipeline behind it.

How to submit

You submit a link, not a file. Same repository as the earlier assessments, new folder.

1. Use your course repository:

`DW-<your roll number>`

2. Create this folder inside it, exactly:

`assignment2/`

3. Put these files in that folder:

File	What it is
`Assignment2_<roll number>.xlsx`	All eight tasks, T8 reading **12 / 12**. A single red cell loses all 8 marks for T8
`pipeline.pbix` — **or** `pipeline.py`	Four sources in, five tables out, every step named, the ladder printed and the twelve checks asserted
`FactShipment_April.csv`	**1,700 rows**
`RejectLog_April.csv`	**35 rows**, every one naming a source, a locator and a reason
`DimCustomer.csv`	**47 rows**, 41 current
`FactShipment_FebToApril.csv`	**2,934 rows**
`answers.md`	**One page in your own words:** the type you chose for each of the six changes, and the one you would defend hardest; the 883,350 kg, and who would have noticed if you had overwritten; the three `-1` counts as numbers, and the sixteen raw values behind them; the cost-per-kilogram sentence you would say to the board; the index you would build, and what it costs you on every load

File	What it is
`README.md`	Your name, roll number, and which tool you used

4. Commit and push:

```
git add assignment2
git commit -m "Assignment 2"
git push
```

5. On the LMS, submit the repository link only:

```
https://github.com/<your-username>/DW-<your roll number>
```

Paste it into the text box. **No attachments.**

> **Commit the `.pbix` to your own repository** — it is the deliverable here, so it belongs in the submission.
>
> If your repository is private the submission cannot be marked. Check your own link in a logged-out browser window before you submit it.

The individual quick check is done on paper in class and is not submitted here.

Definition of done

- ☐ The identity holds: **1,805 = 1,700 + 35 + 70**
- ☐ **17,915,700 kg** and **PKR 158,037,000** for April, exactly
- ☐ **36,036,400 kg** and **PKR 309,882,000** for February to April, exactly
- ☐ All twelve checks pass — **12 / 12**
- ☐ Three credit notes in the fact table, as negatives
- ☐ The three `-1` counts written down **as numbers**, with the unresolved raw values beside them
- ☐ `RejectLog_April` has 35 rows and every one names a source, a locator and a reason
- ☐ `DimCustomer` has 47 rows, 41 current, and `FactShipment_March` still points at the **old** keys
- ☐ February's 34 rows sit on `CustomerKey = -1`, because February had no customer column — **45 rows** carry the Unknown customer in total, and you can say why
- ☐ Punjab, March, still reads **7,566,850 kg**
- ☐ Every transform step **named** — the naming rule has not been repealed
- ☐ Staging queries connection-only; the reject table loads
- ☐ A second refresh produces 1,700 April rows and 47 customer rows, not 3,400 and 53
- ☐ `answers.md` and `README.md` present
- ☐ Pushed, public, and the link opens for a stranger

References

- **Kimball & Ross (2013)**, *The Data Warehouse Toolkit*, 3rd ed. — Ch. 5 on slowly changing dimension Types 0 to 3 (and 4 to 7, if you want to see where this goes), Ch. 15 on aggregate navigation and OLAP, Ch. 19 on ETL subsystem 9, the slowly changing dimension manager.
- **Inmon (2005)**, *Building the Data Warehouse*, 4th ed. — Ch. 4 on the granularity of the data warehouse, and why time-variance is one of the four defining properties rather than a feature.
- **Kimball & Caserta (2004)**, *The Data Warehouse ETL Toolkit* — Ch. 5 on the dimension manager, surrogate key pipelines and the cost of getting a Type 2 loader wrong on the second run.